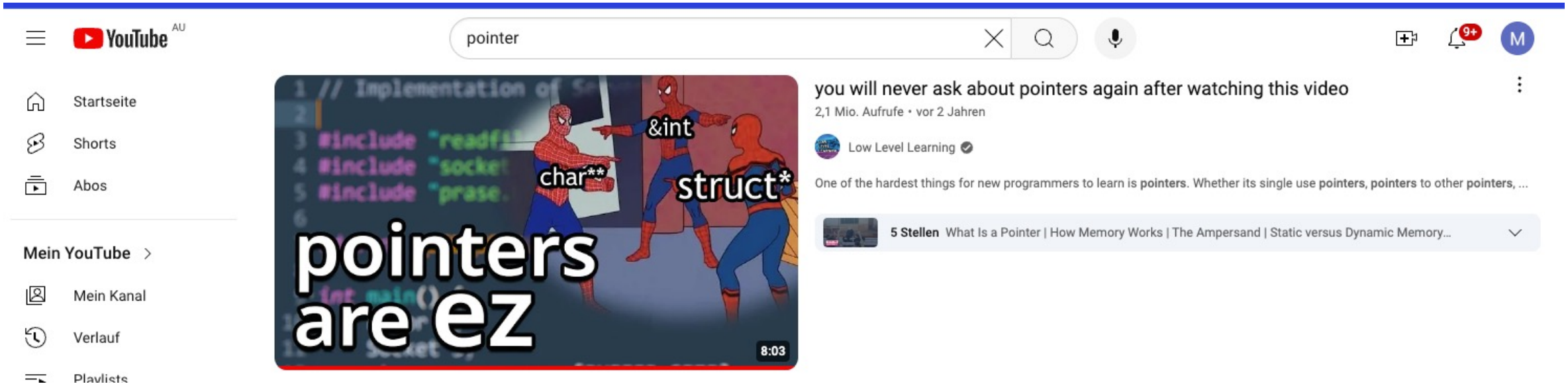
Hello Class!

If you aren’t familiar with **pointers** or would like a quick refresher, please watch the following video:

https://www.youtube.com/watch?v=2ybLD6_2gKM



global variable called *x*

```c
int x = 3;

int main(int argc, char **argv) {
    printf("%d\n", x); // Output 1
    x = 7;
    int x = 4;
    printf("%d\n", x); // Output 2
    x = 8;
    test();
    return 0;
}

void test(void) {
    printf("%d\n", x);
    return;
}
```

update global variable *x*

new local variable called *x*

! this variable now **shadows** the global *x* !

update local variable *x*

access global *x*

Pointers

int cents	define a variable called *cents*
scanf("%d", &cents) the address of the variable *cents*	store the input at the address of *cents*
int *cents	<u>define a pointer</u> called *cents* (not initialised)
int try_one_coin(int *cents) function	<u>defining a function</u> called *try_one_coin*: create a pointer called cents it will point at whatever is passed to the function as input
*cents	<u>using inside a function</u>: access the value stored at the address cents is pointing to

Pointers

```c
#include <stdio.h>

void update_value(int *num) {
    *num = 42;  // Change the value at the memory address num is pointing to
}

int main() {
    int x = 10;
    printf("Before: %d\n", x);

    update_value(&x);  // Pass the address of x

    printf("After: %d\n", x);  // x is now changed
    return 0;
}
```

int cents	define a variable called *cents*
scanf("%d", &cents) the address of the variable *cents*	store the input at the address of *cents*
int *cents	<u>define a pointer</u> called *cents* (not initialised)
int try_one_coin(int *cents) function	<u>defining a function</u> called *try_one_coin*: create a pointer called cents it will point at whatever is passed to the function as input
*cents	<u>using inside a function</u>: access the value stored at the address cents is pointing to

Quicksort

Quicksort

quicksort

```
if n ≤ 1
    return
else
    p ← any element in A[0 ... n − 1]
    (f_e, f_g) ← partition (A, n, p)
    quicksort (A[0 ... f_e − 1])
    quicksort (A[f_g ... n − 1])
```

partition

```
while next < fg
    if A[next] < p
        swap A[fe] and A[next]
        fe, next ← fe + 1, next + 1
    else if A[next] > p
        swap A[next] and A[fg − 1]
        fg ← fg − 1
    else
        next ← next + 1
```

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0 [	**5**	2	7	1	6	**5**	]

$f_e = 0$

$f_g = 5$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

$next, fe, fg \leftarrow 0, 0, n$
while $next < fg$
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	[**5**	2	7	1	6	**5**	]
next=1	[**5**	2	7	1	6	**5**	]

$f_e = 0$

$f_g = 5$

quicksort

if $n \leq 1$

  return

else

  $p \leftarrow any\ element\ in\ A[0\ \ldots n-1]$

  $(f_e, f_g) \leftarrow partition\ (A, n, p)$

  $quicksort\ (A[0\ \ldots f_e - 1])$

  $quicksort\ (A[f_g \ldots n-1])$

partition

$next, fe, fg \leftarrow 0, 0, n$

while $next < fg$

  if $A[next] < p$

    swap $A[fe]$ and $A[next]$

    $fe, next \leftarrow fe + 1, next + 1$

  else if $A[next] > p$

    swap $A[next]$ and $A[fg - 1]$

    $fg \leftarrow fg - 1$

  else

    $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	[**5**	2	7	1	6	**5**	]
next=1	[**5**	2	7	1	6	**5**	]

$f_e = 0$

$f_g = 5$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

$next, fe, fg \leftarrow 0, 0, n$
while $next < fg$
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$
next=3	2	1	**5**	**5**	6	7	$f_e = 2$, $f_g = 4$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow any\ element\ in\ A[0\ \ldots n-1]$
    $(f_e, f_g) \leftarrow partition\ (A, n, p)$
    $quicksort\ (A[0\ \ldots f_e - 1])$
    $quicksort\ (A[f_g \ldots n-1])$

partition

$next, fe, fg \leftarrow 0, 0, n$
while $next < fg$
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$
next=4	2	1	**5**	**5**	6	7	$f_e = 2$, $f_g = 4$

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

next, *fe*, *fg* $\leftarrow 0, 0, n$
while *next* $<$ *fg*
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
next =0	**5**	2	7	1	6	**5**	$f_e = 0$, $f_g = 5$
next=1	**5**	2	7	1	6	**5**	
next=2	2	**5**	7	1	6	**5**	$f_e = 1$, $f_g = 5$
next=3	2	**5**	**5**	1	6	7	$f_e = 1$, $f_g = 4$
next=4	2	1	**5**	**5**	6	7	$f_e = 2$, $f_g = 4$

smaller than pivot

larger than pivot

pivot in correct position 🎉

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow$ *any element in* $A[0 \ \ldots n-1]$
    $(f_e, f_g) \leftarrow$ *partition* (A, n, p)
    quicksort $(A[0 \ \ldots f_e - 1])$
    quicksort $(A[f_g \ldots n-1])$

partition

$next, fe, fg \leftarrow 0, 0, n$
while $next < fg$
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Quicksort

quicksort

if $n \leq 1$
    return
else
    $p \leftarrow any\ element\ in\ A[0\ \ldots n-1]$
    $(f_e, f_g) \leftarrow partition\ (A, n, p)$
    $quicksort\ (A[0\ \ldots f_e - 1])$
    $quicksort\ (A[f_g \ldots n-1])$

partition

while $next < fg$
    if $A[next] < p$
        swap $A[fe]$ and $A[next]$
        $fe, next \leftarrow fe + 1, next + 1$
    else if $A[next] > p$
        swap $A[next]$ and $A[fg - 1]$
        $fg \leftarrow fg - 1$
    else
        $next \leftarrow next + 1$

Case	Time	Example
Best		
Average		
Worst		

Quicksort

quicksort

```
if n ≤ 1
    return
else
    p ← any element in A[0 ... n − 1]
    (f_e, f_g) ← partition (A, n, p)
    quicksort (A[0 ... f_e − 1])
    quicksort (A[f_g ... n − 1])
```

partition

```
while next < fg
    if A[next] < p
        swap A[fe] and A[next]
        fe, next ← fe + 1, next + 1
    else if A[next] > p
        swap A[next] and A[fg − 1]
        fg ← fg − 1
    else
        next ← next + 1
```

Case	Time	Example
Best	$O(n)$	[1,1,1,1,1,1,1]
Average	$O(n \log n)$	[1, 3, 6, 2, 4, 5, 7] picking a pivot that splits the array approximately in half
Worst	$O(n^2)$	[1, 3, 6, 2, 4, 5, 7] picking a pivot that makes a partition of size $n - 1$

Exercise 4 lec05

Exercise 4 lec05

A slightly different approach to *partition* is described by this more relaxed specification:

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$f \leftarrow partition(A, n, p)$
assert: $0 \leq f \leq n-1$ **and** $A[0 \ldots f-1] \leq p$ **and**
$A[f] = p$ **and** $A[f+1 \ldots n-1] \geq p$

4a. Design a function that meets this specification.

4b. Does this approach have any disadvantages or advantages compared to the first one presented?

Exercise 4 lec05

A slightly different approach to *partition* is described by this more relaxed specification:

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$f \leftarrow partition(A, n, p)$
assert: $0 \leq f \leq n-1$ **and** $A[0 \ldots f-1] \leq p$ **and**
$A[f] = p$ **and** $A[f+1 \ldots n-1] \geq p$

(Previous specification)

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$(fe, fg) \leftarrow partition(A, n, p)$
assert: $0 \leq fe < fg \leq n$ **and** $A[0 \ldots fe-1] < p$ **and**
$A[fe \ldots fg-1] = p$ **and** $A[fg \ldots n-1] > p$

Slide 37 of lec05

4a. Design a function that meets this specification.

4b. Does this approach have any disadvantages or advantages compared to the first one presented?

Slide 55 of lec05

Exercise 4 lec05

A slightly different approach to *partition* is described by this more relaxed specification:

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$f \leftarrow partition(A, n, p)$
assert: $0 \leq f \leq n-1$ **and** $A[0 \ldots f-1] \leq p$ **and**
$A[f] = p$ **and** $A[f+1 \ldots n-1] \geq p$

(Previous specification)

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$(fe, fg) \leftarrow partition(A, n, p)$
assert: $0 \leq fe < fg \leq n$ **and** $A[0 \ldots fe-1] < p$ **and**
$A[fe \ldots fg-1] = p$ **and** $A[fg \ldots n-1] > p$

Slide 37 of lec05

4a. Design a function that meets this specification.

4b. Does this approach have any disadvantages or advantages compared to the first one presented?

Slide 55 of lec05

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]

itemFromLeft = 1. item from the left that is larger than the pivot

itemFromRight = 1. item from the left that is larger than the pivot

- swap pivot to the left

Exercise 4 lec05

pivot: p = 5

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]
[5	2	7	1	6	5]

itemFromLeft = 1. item from the left that is larger than the pivot

itemFromRight = 1. item from the left that is larger than the pivot

Exercise 4 lec05

pivot: p = 5

A[0]	A[1]	A[2]	A[3]	A[4]	A[5]
5	2	7	1	6	5

itemFromLeft = 1. item from the left that is larger than the pivot

itemFromRight = 1. item from the left that is larger than the pivot

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]

itemFromLeft = 1. item from the left that is larger than the pivot

itemFromRight = 1. item from the left that is larger than the pivot

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	$\boxed{6}$	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	$\boxed{6}$	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	$\boxed{1}$	$\boxed{6}$	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

- if itemFromRight comes before itemFromLeft we are done and swap the pivot back

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

- if itemFromRight comes before itemFromLeft we are done and swap the pivot back

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

smaller than pivot larger than pivot

itemFromLeft = 1. item from the left that is larger than the pivot (or equal)

itemFromRight = 1. item from the right that is smaller than the pivot (or equal)

- if itemFromRight comes before itemFromLeft we are done and swap the pivot back

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

smaller than pivot larger than pivot

pseudocode

$identify\ \ x\ \ such\ that\ A[x] = p$

$swap\ (A[0], A[x])$

$le, ge\ \leftarrow 1, n-1$ *// le = less than or equal (itemFromRight),*
ge = greater than or equal (itemFromLeft)

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

smaller than pivot larger than pivot

pseudocode

$identify\ \ x\ \ such\ that\ A[x] = p$

$swap\ (A[0], A[x])$

$le, ge\ \leftarrow 1, n-1$ *// le = less than or equal (itemFromRight), ge = greater than or equal (itemFromLeft)*

while $le \leq ge$

  while $le < n$ **and** $A[le] < p$

    $le\ \leftarrow le + 1$

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

smaller than pivot larger than pivot

pseudocode

$identify\ \ x\ \ such\ that\ A[x] = p$

$swap\ (A[0], A[x])$

$le, ge\ \leftarrow 1, n-1$ *// le = less than or equal (itemFromRight), ge = greater than or equal (itemFromLeft)*

while $le \leq ge$

  while $le < n$ **and** $A[le] < p$

    $le\ \leftarrow le + 1$

  while $A[ge] > p$

    $ge\ \leftarrow ge - 1$

Exercise 4 lec05

pivot: p = 5

	A[0]	A[1]	A[2]	A[3]	A[4]	A[5]	
[	5	2	7	1	6	5	]
[	5	2	5	1	6	7	]
[	1	2	5	5	6	7	]

smaller than pivot larger than pivot

pseudocode

$identify\ \ x\ \ such\ that\ A[x] = p$

$swap\ (A[0], A[x])$

$le, ge\ \leftarrow 1, n-1$ *// le = less than or equal (itemFromRight), ge = greater than or equal (itemFromLeft)*

while $le \leq ge$

  while $le < n$ **and** $A[le] < p$

    $le\ \leftarrow le + 1$

  while $A[ge] > p$

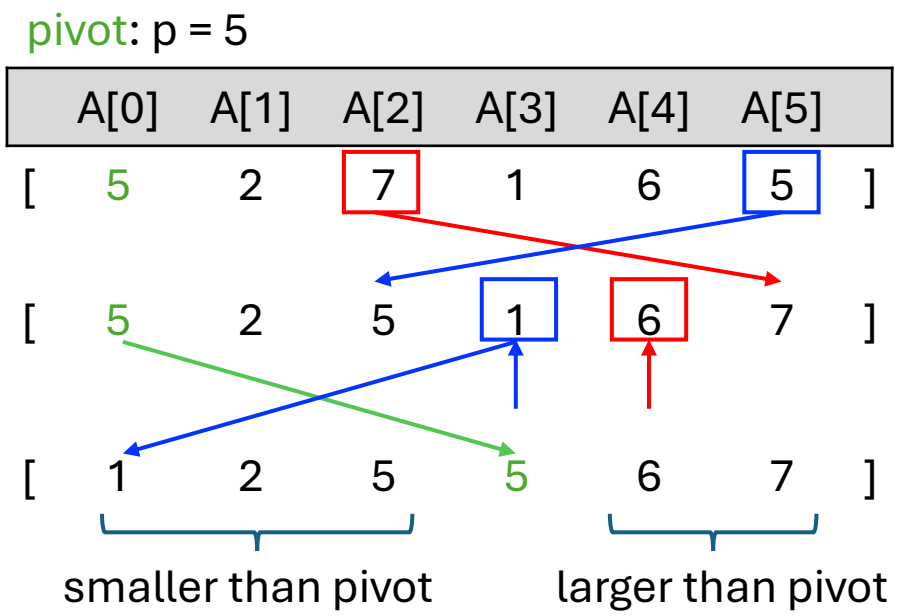
    $ge\ \leftarrow ge - 1$

  if $le \leq ge$

    $swap(A[le], A[ge])$

    $le, ge\ \leftarrow le + 1, ge - 1$

Exercise 4 lec05



pseudocode

$identify\ \ x\ \ such\ that\ A[x] = p$

$swap\ (A[0], A[x])$

$le, ge\ \leftarrow 1, n-1$ *// le = less than or equal (itemFromRight), ge = greater than or equal (itemFromLeft)*

while $le \leq ge$

  while $le < n$ **and** $A[le] < p$

    $le\ \leftarrow le + 1$

  while $A[ge] > p$

    $ge\ \leftarrow ge - 1$

  if $le \leq ge$

    $swap(A[le], A[ge])$

    $le, ge\ \leftarrow le + 1, ge - 1$

$swap(A[0], A[ge])$

return ge

Exercise 4 lec05

assert: $n > 1$ **and** $p \in A[0 \ldots n-1]$
$f \leftarrow partition(A, n, p)$
assert: $0 \leq f \leq n-1$ **and** $A[0 \ldots f-1] \leq p$ **and**
$A[f] = p$ **and** $A[f+1 \ldots n-1] \geq p$

pseudocode

```
identify x such that A[x] = p
swap (A[0], A[x])
le, ge ← 1, n − 1    // le = less than or equal (itemFromRight),
                     ge = greater than or equal (itemFromLeft)
while le ≤ ge
    while le < n and A[le] < p
        le ← le + 1
    while A[ge] > p
        ge ← ge − 1
    if le ≤ ge
        swap(A[le], A[ge])
        le, ge ← le + 1, ge − 1
swap(A[0], A[ge])
return ge
```

Exercise 4 lec05

Advantages:

- Fewer element swaps than the three-way partitioning

Disadvantages:

- Still leads to $\mathrm{O}(n \log n)$ time on arrays with repeated elements whereas three-way partitioning gives $\mathrm{O}(n)$ time on arrays and sub-arrays in which all n items are the same

Case	Time	Example
Best	$O(n \log n)$	[1,1,1,1,1,1,1]
Average	$O(n \log n)$	[1, 3, 6, 2, 4, 5, 7] picking a pivot that splits the array approximately in half
Worst	$O(n^2)$	[1, 3, 6, 2, 4, 5, 7] picking a pivot that makes a partition of size $n - 1$